Lab Submission for Nov 2 2026

By Muhammad Junaid Ali (210163492)

Submission Details/Overview

Submission Details

Submission Instructions, according to 'CS3CI Lab Sign-Off Brief' on Blackboard:

" ... submit the source code for the novel parts of the algorithms used in tasks 1-3 as a single doc, pdf or rtf file by the stated deadline. You only need to submit the novel parts themselves (e.g. if you implemented a new mutation operator for an EA, you would submit the code relating to that, not your entire implementation). You are allowed to base the non-novel parts of your implementation on work you have done with your classmates, on code that your tutor has provided or even on libraries that you have found online. The implementation of the novel parts of your algorithm should be solely your own work, however. "

Task Description, same source: " For tasks 1-3 (described below) you are asked to show a novel variant of an algorithm. For the purposes of this assignment, this means a variant of that algorithm which is in some (non-trivial) way different from the algorithms you were given step-by-step instructions to create in the module's practical tasks. Guidance on ways in which you could modify your algorithms to create a novel variant is given at the end of practical tasks 2, 3 and 4. "

Task Overview

The tasks you will be asked to perform are:

1. Show your novel variant of local search running on the TSP (based on Practical 2). Each time your local search finds an improved route, your implementation should print it out with its cost.

This means submission must include novel parts of Practical 2

Each time algorithm finds an improved route it should print it out along with its cost

Time-based termination = 10 seconds, other termination= = no more than 30

2. Show your novel variant of a computational intelligence algorithm of your choice running on the TSP (potentially based on Practical 3). Each time your algorithm finds an improved route, your implementation should print it out with its cost. You should set the termination condition such that the algorithm takes no more than 30 seconds to run.

TSP compatible Labs are Labs 2 and 4 for novel variants

Each time algorithm finds an improved route it should print it out along with its cost

CONFIRM TESTS WILL WORK WITH .CSV FILE - Lab 4 working for .CSV

3. Show your novel variant of a computational intelligence algorithm of your choice implemented on the antenna array design problem (potentially based on Practical 4). You will be provided with a number of antennae and an angle as values to use in the sign-off. Each time your algorithm finds an improved solution, your implementation should print it out with its cost. As above, you should set the termination condition such that the algorithm takes no more than 30 seconds to run.

Lab 3 applicable to 'antennae and an angle'

Needs to print solution and cost every updated best_ssl

4. For one of your novel variants, identify an academic paper or textbook as the source of the change you made to the baseline algorithm. You should be able to display the paper/textbook section to your assessor and show where in the paper/textbook the idea that you used is described. You should take no more than 1 minute to explain this.

Can use Lab 3

(https://ieeexplore.ieee.org/document/5171070)

5. Shortly before your sign-off appointment, you will also be provided with a new, unseen csv file, formatted in the same way as those given for practical task 5, giving the results of applying to algorithms, A and B, to the same problem. Show whether these results are sufficient to claim that algorithm A performs statistically significantly better on the problem than algorithm B does at significance level $\alpha = 0.05$. Please prepare your analysis before your sign-off slot such that you can demonstrate your findings in no more than 1 minute.

How this NoteBook will be organised

First, there will be a section that holds ONLY the novel sections of each lab, then I will also add a section for each lab that will have fully operational novel variants from each lab

Novel ONLY Section

Lab 2

These are the parts of my Lab 2 variation that differentiate it from the original lab tasks.

I found tabu search from (https://www.geeksforgeeks.org/dsa/what-is-tabu-search/)

This variant uses a 'tabu_list' to keep track of previous solutions and a neighbourhood to produce nearby solutions, solutions within tabu list are not produced again.

In [31]:
```
def tabu_search(tsp_instance, target_time):

    start_time = time.time()
    current_route = random_route(tsp_instance)
    best_route = copy.deepcopy(current_route)
    best_distance = evaluate(tsp_instance, best_route)
    tabu_list = []
    max_tabu_length = 5

    def best_tabu_neighbour_step(current_route, neighbourhood):

            best_neighbour = None
            best_neighbour_cost = float('inf')
            best_move = None

            for neighbour in neighbourhood:
                for i in range(1, len(current_route) - 1):
                    for j in range(i + 1, len(current_route) - 1):
                        if (current_route[i] == neighbour[j] and current_route[j
                            move = (i, j)
                            break
                        else:
                            continue
                    break

                neighbour_cost = evaluate(tsp_instance, neighbour)

                if move in tabu_list and neighbour_cost >= best_distance:
                    continue

                # Find the best allowed neighbour
                if neighbour_cost < best_neighbour_cost:
                    best_neighbour = neighbour
                    best_neighbour_cost = neighbour_cost
                    best_move = move

            return best_neighbour, best_neighbour_cost, best_move

    # within main loop
    if len(tabu_list) > max_tabu_length:
        tabu_list.pop(0)
```

Lab 3

I have used multiple equations from a paper I found (https://ieeexplore.ieee.org/document/5171070)

'k' is from 'k = |f(t)−f(t−T)| / |f(t−T)|' where t = current_fitness, T = previous_fitness (by 5 runs), k = relative change in fitness

w = { α1 + r/2, k≥0.05 { α2 + r/2, k<0.05 where a1 = 0.6 and a2 = 0.2 and r is a uniformly distributed random number in the interval [0,1]

when α1 = 0.6, 1.1 ≥ w ≥ 0.6 when α2 = 0.2, 0.7 ≥ w ≥ 0.2

In [32]:
```python
# Variant of PSO with Adaptive Inertia Weight
# Inspiration from : https://ieeexplore.ieee.org/document/5171070 <- will be cal
# Will not be taking the 'Mutation' aspect from the paper
import random

def calculate_inertia_weight(self):
        if len(self.fitness_history) <= self.T:
            k = 0
        else:
            current_fitness = self.fitness_history[-1]
            # -(self.T + 1), gets fitness from T iterations ago, uses + 1 to acc
            previous_fitness = self.fitness_history[-(self.T + 1)]
            # Equation from Paper: k = |f(t)-f(t-T)| / |f(t-T)|
            # 1e-12 is used to avoid /0 #
            k = (
                abs(current_fitness - previous_fitness)
                / max(abs(previous_fitness), 1e-12)
            )

        if k >= self.threshold:
            alpha = self.alpha1
        else:
            alpha = self.alpha2

        # Paper shows w as:
        # w = { α1 + r/2, k≥0.05
        #     { α2 + r/2, k<0.05
        # "where, r is a uniformly distributed random number in the interval [0,
        w = alpha + random.uniform(0, 1) / 2

        self.inertia_history.append(w)
        self.k_history.append(k)

        return w
```

Lab 4

Copy-pasted out of my lab notebook, for Lab 4's novel idea I switched from using a 'swap mutation' (where 2 random cities are swapped within a route) and instead made an 'inversion mutation' this overrides the previous mutation and will now select a section of the route and reverse its order only within the selected section.

The mutate method was also overridden so that it would call inversion mutation instead of swap mutation.

Only requires random for mutate probability functionality

In [33]:
```
import random

def __inversion_mutation(self, route):
    mutated = route.copy()
    start, end = sorted(random.sample(range(1, len(route)), 2))
    mutated[start:end + 1] = reversed(mutated[start:end + 1])
    return mutated

def mutate(self, route, mutation_prob=0.7):
    if random.random() < mutation_prob:
        return self.__inversion_mutation(route)

    return route.copy()
```

Fully Operational Novel Sections

tsp.py from lab1code.zip for Lab 2

In [34]:
```
import numpy as np
import math
import matplotlib.pyplot as plt
import networkx as nx
from networkx.classes.function import path_weight
import random

# Constructs a TSP instance from a matrix (i.e. list of lists) adjacency_matrix
# in which adjacency_matrix[i][j] corresponds to the length of the edge from
# city i to city j.
def construct(adjacency_matrix):
  return nx.from_numpy_array(np.matrix(adjacency_matrix))

# Constructs a TSP instance from a matrix city_coords in which each row of the
# matrix represents the Cartesian coordinates of a city.
def construct_from_city_coords(city_coords):
  def dist(x,y):
    return math.sqrt((x[0] - y[0])**2 + (x[1] - y[1])**2)

  adjacency_matrix = [ [dist(x,y) for x in city_coords] for y in city_coords]

  return construct(adjacency_matrix)

# Displays a TSP instance on a 2D plane. Optional args:
#  - city_coords: Cartesian coordinates of the cities use to position cities in
#    the 2D plane. If no coordinates are provided, arbitrary ones are
#    generated.
#  - label_edges: Whether or not the edges of the graph should be labelled with
#    their weights. It is recommended to set this as False for all but very
#    small graphs as the number of edges quickly grows very large.
#  - title: A title for the plot
def display_instance(tsp_instance,city_coords = None,label_edges=False,title=Non
```

```
  if city_coords is None:
    city_coords = nx.spring_layout(tsp_instance)

  # First, we create a new figure into which to plot our graph.
  plt.figure()
  # With the positions defined, we can now draw the graph itself...
  nx.draw_networkx(tsp_instance,city_coords)
  # ... and, if we wish, we can label the edges according to their weights.
  if label_edges:
    edge_labels = nx.get_edge_attributes(tsp_instance,'weight')
    nx.draw_networkx_edge_labels(tsp_instance,city_coords,edge_labels=edge_label
  # Add a title to the plot
  if title is not None:
    plt.title(title)
  # Finally, we display the graph
  plt.show()

# Checks that an array specifies a valid TSP route. A valid TSP route should:
#   - visit all cities
#   - visit its starting city twice: once at the start and once at the end of
#     the route,
#   - aside from its starting city, it should visit each city exactly once.
#
# An example of a valid route for a TSP instance with 4 cities would be
# [1,2,3,0,1].
def is_valid_route(tsp_instance,route):
  number_of_cities = tsp_instance.number_of_nodes()
  # Check if the route returns to the starting city
  if route[0] != route[-1]: return False
  # Check that only the starting city is repeated in the route
  elif len(route) != len(set(route)) + 1: return False
  # Check all cities have been visited
  elif len(route) != number_of_cities + 1: return False
  # If so, the route is valid
  else: return True

# Displays a TSP instance on a 2D plane. Overlays a specified route on this
# instance. Optional args:
#  - city_coords: Cartesian coordinates of the cities use to position cities in
#    the 2D plane. If no coordinates are provided, arbitrary ones are
#    generated.
#  - label_edges: Whether or not the edges of the graph should be labelled with
#    their weights. It is recommended to set this as False for all but very
#    small graphs as the number of edges quickly grows very large.
#  - title: A title for the plot
def display_route(tsp_instance,route,city_coords = None,label_edges=False,title=
  if not is_valid_route(tsp_instance,route):
    raise ValueError('{} is not a valid route for a TSP instance with {} cities'

  if city_coords is None:
    city_coords = nx.spring_layout(tsp_instance)

  # First, we create a new figure into which to plot our graph.
  plt.figure()
  # With the positions defined, we can now draw the TSP instance itself...
  nx.draw_networkx(tsp_instance,city_coords,edge_color="blue",width=0.5)

  # ...then overlay the route.
  edge_list = list(nx.utils.pairwise(route))
  nx.draw_networkx(
```

```
    tsp_instance,
    city_coords,
    edgelist=edge_list,
    edge_color="red",
    width=3,
  )
  # Label the edges according to their weights.
  if label_edges:
    edge_labels = nx.get_edge_attributes(tsp_instance,'weight')
    nx.draw_networkx_edge_labels(tsp_instance,city_coords,edge_labels=edge_label
  # Add a title to the plot
  if title is not None:
    plt.title(title)
  # Finally, we display the graph
  plt.show()

# Evaluates a given route on a TSP instance, returning the total length of the
# tour.
def evaluate(tsp_instance,route):
  if not is_valid_route(tsp_instance,route):
    raise ValueError('{} is not a valid route for a TSP instance with {} cities'
  return path_weight(tsp_instance, route, weight="weight")

# Generates a random, valid route for a given TSP instance. As TSP routes are
# complete loops, we will generate routes which start and end at city 0.
def random_route(tsp_instance):
  number_of_cities = tsp_instance.number_of_nodes()
  # Start with a valid route (minus the starting city - city 0).
  route_middle = [*range(1,number_of_cities)]
  # Shuffle the route into a random order.
  random.shuffle(route_middle)
  # Add the starting city and loop back to it at the end.
  return [0] + route_middle + [0]
```

Lab 2

Lab 2 requires tsp.py cell to be ran first

In [35]:

```
################################################################################
# Step 6
################################################################################

# Novel Variant : TABU search (https://www.geeksforgeeks.org/dsa/what-is-tabu-se

import copy
import time
import pandas as pd

def city_swap_neighbourhood(route):
    # Randomly select two cities to swap, cannot select city index[0] or [length
    # Return a neighbourhood of all possible routes with the two cities swapped

    neighbourhood = []

    for i in range(1, len(route) - 1):
        for j in range(i + 1, len(route) - 1):
            new_route = copy.deepcopy(route)
```

```
            new_route[i], new_route[j] = new_route[j], new_route[i]

            neighbourhood.append(new_route)

    return neighbourhood

def tabu_search(tsp_instance, target_time):

    start_time = time.time()
    current_route = random_route(tsp_instance)
    best_route = copy.deepcopy(current_route)
    best_distance = evaluate(tsp_instance, best_route)
    tabu_list = []
    max_tabu_length = 5

    iteration = 0

    print(f"Iteration {iteration}:")
    print(f"Cost: {best_distance}")
    print(f"Route: {best_route}")

    # modified best_neighbour_step to account for tabu list and best overall sol
    def best_tabu_neighbour_step(current_route, neighbourhood):

        best_neighbour = None
        best_neighbour_cost = float('inf')
        best_move = None

        for neighbour in neighbourhood:
            for i in range(1, len(current_route) - 1):
                for j in range(i + 1, len(current_route) - 1):
                    if (current_route[i] == neighbour[j] and current_route[j] ==
                        move = (i, j)
                        break
                    else:
                        continue
                break

            neighbour_cost = evaluate(tsp_instance, neighbour)

            if move in tabu_list and neighbour_cost >= best_distance:
                continue

            # Find the best allowed neighbour
            if neighbour_cost < best_neighbour_cost:
                best_neighbour = neighbour
                best_neighbour_cost = neighbour_cost
                best_move = move

        return best_neighbour, best_neighbour_cost, best_move

    # starting main loop
    while time.time() - start_time < target_time:

        iteration += 1
        neighbourhood = city_swap_neighbourhood(current_route)
        best_neighbour, best_neighbour_cost, best_move = best_tabu_neighbour_ste
            current_route, neighbourhood)

        if best_neighbour is None:
```

```
            break

        current_route = copy.deepcopy(best_neighbour)

        tabu_list.append(best_move)

        if len(tabu_list) > max_tabu_length:
            tabu_list.pop(0)

        if best_neighbour_cost < best_distance:
            best_route = copy.deepcopy(best_neighbour)
            best_distance = best_neighbour_cost

            print(f"\nIteration {iteration}:")
            print(f"Cost: {best_distance}")
            print(f"Route: {best_route}")

    return best_route, best_distance

ulysses16_coords = pd.read_csv('ulysses16.csv',header=0).values
ulysses16_tsp_instance = construct_from_city_coords(ulysses16_coords)

df = pd.read_csv('ulysses16.csv')

num_cities = len(df)

target_time = 10

t1 = tabu_search(ulysses16_tsp_instance, target_time)
# t2 = tabu_search(ulysses16_tsp_instance, target_time + 10)
# t3 = tabu_search(ulysses16_tsp_instance, target_time + 20)
print("Final Route Found:")
print(f"Best Route (and Cost): {t1}")
# print(f"t2: {t2}")
# print(f"t3: {t3}")
```

```
Iteration 0:
Cost: 131.61504550938326
Route: [0, 11, 8, 1, 15, 7, 6, 2, 4, 10, 5, 9, 13, 14, 12, 3, 0]

Iteration 1:
Cost: 103.5416476593875
Route: [0, 11, 2, 1, 15, 7, 6, 8, 4, 10, 5, 9, 13, 14, 12, 3, 0]

Iteration 2:
Cost: 90.36140474235567
Route: [0, 3, 2, 1, 15, 7, 6, 8, 4, 10, 5, 9, 13, 14, 12, 11, 0]

Iteration 3:
Cost: 81.22769226585568
Route: [0, 3, 2, 1, 15, 7, 6, 5, 4, 10, 8, 9, 13, 14, 12, 11, 0]

Iteration 4:
Cost: 78.50796528587661
Route: [0, 3, 2, 1, 15, 11, 6, 5, 4, 10, 8, 9, 13, 14, 12, 7, 0]

Iteration 5:
Cost: 77.45560548808326
Route: [0, 3, 1, 2, 15, 11, 6, 5, 4, 10, 8, 9, 13, 14, 12, 7, 0]

Iteration 6:
Cost: 76.85857314977021
Route: [0, 3, 1, 2, 15, 11, 6, 5, 4, 10, 8, 9, 14, 13, 12, 7, 0]

Iteration 7:
Cost: 76.15347838271309
Route: [0, 3, 1, 2, 15, 11, 6, 5, 4, 10, 8, 9, 12, 13, 14, 7, 0]

Iteration 27:
Cost: 75.77481299129495
Route: [0, 7, 3, 1, 2, 15, 14, 5, 4, 10, 8, 9, 6, 13, 11, 12, 0]

Iteration 28:
Cost: 75.70300971051522
Route: [0, 7, 3, 1, 2, 15, 14, 5, 4, 10, 8, 9, 6, 13, 12, 11, 0]

Iteration 29:
Cost: 75.47709668138731
Route: [0, 7, 3, 1, 2, 15, 14, 5, 4, 10, 8, 9, 6, 11, 12, 13, 0]

Iteration 120:
Cost: 75.30255099967557
Route: [0, 7, 3, 2, 1, 15, 13, 14, 4, 10, 8, 9, 6, 5, 11, 12, 0]

Iteration 121:
Cost: 74.25019120188223
Route: [0, 7, 3, 1, 2, 15, 13, 14, 4, 10, 8, 9, 6, 5, 11, 12, 0]

Iteration 122:
Cost: 74.15356685766439
Route: [0, 7, 3, 1, 2, 15, 13, 14, 4, 10, 8, 9, 5, 6, 11, 12, 0]
Final Route Found:
Best Route (and Cost): ([0, 7, 3, 1, 2, 15, 13, 14, 4, 10, 8, 9, 5, 6, 11, 12,
0], 74.15356685766439)
```

Antenna Array for Lab 3

In [36]:
```
import math
import numpy as np

# Car price prediction problem
class AntennaArray:
    # The minimum distance between antennae.
    MIN_SPACING = 0.25

    # Construct an antenna design problem instance.
    # The parameter "n_antennae" specifies the number of antennae in our array.
    # The parameter "steering_angle" specifies the desired direction of the
    # main beam in degrees.
    def __init__(self,n_antennae,steering_angle):
      self.n_antennae = n_antennae
      self.steering_angle = steering_angle

    # Rectangular bounds on the search space.
    # Returns a 2D array b such that b[i][0] is the minimum permissible value
    # of the ith solution component and b[i][1] is the maximum.
    def bounds(self):
        return [[0,self.n_antennae/2] for i in range(self.n_antennae)]

    # Check whether an antenna design lies within the problem's feasible
    # region.
    # A design is a vector of n_antennae anntena placements.
    # A placement is a distance from the left hand side of the antenna array.
    # A valid placement is one in which
    #   1) all antennae are separated by at least MIN_SPACING
    #   2) the aperture size (the maximum element of the array) is exactly
    #      n_antennae/2.
    def is_valid(self, design):
        # The design has the correct number of antennae
        if len(design) != self.n_antennae:
          return False

        des = design.copy()
        des.sort()

        # Apperture size is exactly n_antennae/2
        if abs(des[-1] - self.n_antennae/2) > 1e-10:
          return False
        # All antennae lie within the problem bounds
        for placement, bound in zip(des,self.bounds()):
          if placement < bound[0] or placement > bound[1]:
            return False

        # All antennae are separated by at least MIN_SPACING
        for i in range(len(des)-1):
          if des[i+1] - des[i] < AntennaArray.MIN_SPACING:
            return False

        # If none of the above checks have been failed, the design must be
        # valid.
        return True

    # Evaluate an antenna design returning peak SSL.
```

```
    # Designs which violate problem constraints will be penalised with extremely
    # high costs.
    # The parameter "design" is a valid antenna array design.
    # If the design is not valid, it will be penalised by being assigned the
    # maximum possible numerical cost, float('inf').
    def evaluate(self,design):
        if(not self.is_valid(design)): return float('inf');

        class PowerPeak:
            def __init__(self,elevation,power):
                self.elevation = elevation;
                self.power = power;

        # Find all the peaks in power
        peaks = []
        prev = PowerPeak(0.0,float('-inf'))
        current = PowerPeak(0.0,self.__array_factor(design,0.0));
        for elevation in np.arange(0.01,180.01,0.01):
            nxt = PowerPeak(elevation,self.__array_factor(design,elevation))
            if current.power >= prev.power and current.power >= nxt.power:
                peaks.append(current)
            prev = current
            current = nxt
        peaks.append(PowerPeak(180.0,self.__array_factor(design,180.0)))
        peaks.sort(reverse = True,key = lambda peak:peak.power)

        # No side-lobes case
        if len(peaks)<2:
          return float('-inf')

        # Filter out main lobe and then return highest lobe level
        distance_from_steering = abs(peaks[0].elevation - self.steering_angle);
        for i in range(1,len(peaks)):
          # If the peak with the highest power is not the closest peak to the
          # steering angle, it is a side lobe and, therefore, must be the peak
          # side lobe.
          if abs(peaks[i].elevation - self.steering_angle) < distance_from_steer
            return peaks[0].power
        # Otherwise, the peak with the highest power is not a side lobe. The
        # peak with the second highest power must be the peak side lobe.
        return peaks[1].power

    def __array_factor(self,design,elevation):
        steering = 2.0*math.pi*self.steering_angle/360.0;
        elevation = 2.0*math.pi*elevation/360.0;
        sum = 0.0
        for placement in design:
            sum += math.cos(2 * math.pi * placement * (math.cos(elevation) - mat
        return 20.0*math.log(abs(sum))
```

Lab 3

Lab 3 requires AntennaArray Method given by Lecturer to run.

In [37]:
```
# Step 6

# Variant of PSO with Adaptive Inertia Weight
# Inspiration from : https://ieeexplore.ieee.org/document/5171070 <- will be cal
```

```python
# Will not be taking the 'Mutation' aspect from the paper

import random

def constrained_random_parameters(antenna_array_problem):
    b = antenna_array_problem.bounds()
    design = [low + random.random()*(high-low) for [high, low] in b]
    design[-1] = antenna_array_problem.n_antennae/2
    return design

def particle(antenna_array_problem):
    # Initialise the particle's position and velocity
    position = constrained_random_parameters(antenna_array_problem)

    velocity = [
        random.uniform(-1, 1)
        for _ in range(antenna_array_problem.n_antennae)
    ]

    # Evaluate the initial position
    best_position = position.copy()
    best_sll = antenna_array_problem.evaluate(position)

    return {
        'position': position,
        'velocity': velocity,
        'best_position': best_position,
        'best_sll': best_sll
    }

def update_particle(particle, antenna_array_problem, global_best_position, w=0.7

    # Store the previous position for constraint handling
    previous_position = particle['position'].copy()

    # Update velocity
    for i in range(antenna_array_problem.n_antennae):

        r1 = random.random()
        r2 = random.random()

        particle['velocity'][i] = (
            w * particle['velocity'][i]
            + c1 * r1 * (particle['best_position'][i] - particle['position'][i])
            + c2 * r2 * (global_best_position[i] - particle['position'][i])
        )

    # Update position
    new_position = [
        particle['position'][i] + particle['velocity'][i]
        for i in range(antenna_array_problem.n_antennae)
    ]

    # Constraint handling: invisible wall strategy
    if not antenna_array_problem.is_valid(new_position):
        particle['position'] = previous_position
        particle['velocity'] = [0] * antenna_array_problem.n_antennae
        return

    # Accept the new position
```

```
    particle['position'] = new_position

    # Evaluate the new position
    new_sll = antenna_array_problem.evaluate(new_position)

    # Update personal best if the new solution is better
    if new_sll < particle['best_sll']:
        particle['best_position'] = new_position.copy()
        particle['best_sll'] = new_sll

class AdaptiveSwarm():
    def __init__(
            self,
            antenna_array_problem,
            num_particles,
            T=5,
            # From Paper: "Generally, α1 and α2 are taken respectively 0.6 and 0
            alpha1=0.6, # when α1 = 0.6, 1.1 ≥ w ≥ 0.6
            alpha2=0.2, # when α2 = 0.2, 0.7 ≥ w ≥ 0.2
            threshold=0.05
    ):
        # Additions
        self.T = T
        self.alpha1 = alpha1
        self.alpha2 = alpha2
        self.threshold = threshold

        self.antenna_array_problem = antenna_array_problem
        self.particles = [particle(antenna_array_problem) for _ in range(num_par
        self.num_particles = num_particles
        best_particle = min(
            self.particles,
            key=lambda p: p['best_sll']
        )
        self.global_best_position = best_particle['best_position']
        self.global_best_sll = best_particle['best_sll']

        self.fitness_history = [self.global_best_sll]
        self.inertia_history = []

        # relative fitness change
        self.k_history = []

    def calculate_inertia_weight(self):
        if len(self.fitness_history) <= self.T:
            k = 0
        else:
            current_fitness = self.fitness_history[-1]
            # -(self.T + 1), gets fitness from T iterations ago, uses + 1 to acc
            previous_fitness = self.fitness_history[-(self.T + 1)]
            # Equation from Paper: k = |f(t)-f(t-T)| / |f(t-T)|
            # 1e-12 is used to avoid /0 #
            k = (
                abs(current_fitness - previous_fitness)
                / max(abs(previous_fitness), 1e-12)
            )

        if k >= self.threshold:
            alpha = self.alpha1
        else:
```

```
            alpha = self.alpha2

        # Paper shows w as:
        # w = { α1 + r/2, k≥0.05
        #     { α2 + r/2, k<0.05
        # "where, r is a uniformly distributed random number in the interval [0,
        w = alpha + random.uniform(0, 1) / 2

        self.inertia_history.append(w)
        self.k_history.append(k)

        return w

    def iterate(self, c1=1.1193, c2= 1.1193):
        w = self.calculate_inertia_weight()

        for p in self.particles:
            update_particle(
                p,
                self.antenna_array_problem,
                self.global_best_position,
                w=w,
                c1=c1,
                c2=c2
            )
            if p['best_sll'] < self.global_best_sll:
                self.global_best_position = p['best_position'].copy()
                self.global_best_sll = p['best_sll']

        self.fitness_history.append(self.global_best_sll)

    def get_best_solution(self):
            return {
                'position': self.global_best_position.copy(),
                'sll': self.global_best_sll
            }
```

In [38]:

```
antenna_array_problem = AntennaArray(10, 90)

adaptive_swarm = AdaptiveSwarm(
    antenna_array_problem,
    num_particles=20,
    T=5
)

n_iterations = 75

for iteration in range(n_iterations):

    previous_best_sll = adaptive_swarm.global_best_sll

    adaptive_swarm.iterate()



    # uncomment for per iter values
    # print(
    #     f"Iteration {iteration + 1}: "
    #     f"Best SLL = {adaptive_swarm.global_best_sll:.4f}, "
    #     f"k = {adaptive_swarm.k_history[-1]:.4f}, "
```

```
    #     f"w = {adaptive_swarm.inertia_history[-1]:.4f}"
    # )

    if adaptive_swarm.global_best_sll < previous_best_sll:
        print(f"\nIteration {iteration + 1}:")
        print(f"Best SLL: {adaptive_swarm.global_best_sll:.6f}")
        print(f"Best Position: {adaptive_swarm.global_best_position}")

print("\nFinal best solution:")
print(adaptive_swarm.get_best_solution())
```

```
Iteration 49:
Best SLL: 31.810413
Best Position: [1.2519034656386832, 3.0013900885740825, 2.0103171336502648, 1.713
3932054892007, 4.223823618417287, 2.517126254529761, 3.8600877095521584, 4.586386
080234687, 3.266510436101991, 5.0]

Iteration 71:
Best SLL: 24.569192
Best Position: [2.296481813805201, 3.339734483765281, 1.9956423138773007, 1.58592
2455282401, 3.784920643378549, 2.8587336640403755, 0.907940913434022, 2.584871977
3617534, 1.1647724603402807, 5.0]

Final best solution:
{'position': [2.296481813805201, 3.339734483765281, 1.9956423138773007, 1.5859224
55282401, 3.784920643378549, 2.8587336640403755, 0.907940913434022, 2.58487197736
17534, 1.1647724603402807, 5.0], 'sll': 24.569192213747826}
```

Lab 4

In [39]:

```
# Step 7

import random
import time
import pandas as pd
import numpy as np

class InverseMutationTSP():
        # Variation changes __swap_mutation for __inversion_mutation
        # New mutation inverts cities within selected area
        def __init__(self, num_cities, min_cost=1, max_cost=100):
            self.adj_matrix = self.build_adj_matrix(num_cities)
            self.num_cities = num_cities
            self.adj_matrix = self.random_tsp(min_cost, max_cost)
            self.population = []

        def build_adj_matrix(self, num_cities):
            return [
                [0 for _ in range(num_cities)]
                for _ in range(num_cities)
            ]

        def print_matrix(self):
            for _ in self.adj_matrix:
                print(_)
            print()

        def random_route(self):
            route = list(range(1, self.num_cities))
```

```
        random.shuffle(route)

        return [0] + route

    # Step 3: Population Generation
    def populate(self, pop_size):
        self.population = []
        for _ in range(pop_size):
            route = self.random_route()
            self.population.append(route)
        return self.population

    def print_population(self):
        for route in self.population:
            print(route, "Cost:", self.evaluate_route(route))
        print()

    def evaluate_route(self, route):
        total_cost = 0
        num_cities = len(route)

        for i in range(num_cities):
            from_city = route[i]
            to_city = route[(i + 1) % num_cities]  # '% num_cities' makes ro
            total_cost += self.adj_matrix[from_city][to_city]

        return total_cost

    def random_tsp(self, min_cost=1, max_cost=100):

        print("Generating Routes ...")
        for i in range(self.num_cities):
            for j in range(i + 1, self.num_cities):
                cost = random.randint(min_cost, max_cost)
                self.adj_matrix[i][j] = cost
                self.adj_matrix[j][i] = cost

        return self.adj_matrix

    # Step 4: Parent Selection
    def select_parents(self, num_offspring):
        tournament_size = 2
        if len(self.population) < tournament_size:
            raise ValueError(f"Population must have at least {tournament_siz
        elif len(self.population) != num_offspring:
            raise ValueError("Population size and Number of Offspring must b

        parents = []
        for _ in range(num_offspring):
            tournament = random.sample(self.population, tournament_size)
            winner = min(tournament, key=self.evaluate_route)
            parents.append(winner)

        return parents

    def __inversion_mutation(self, route):
        mutated = route.copy()
        start, end = sorted(random.sample(range(1, len(route)), 2))
        mutated[start:end + 1] = reversed(mutated[start:end + 1])
        return mutated
```

```
    def mutate(self, route, mutation_prob=0.7):
        if random.random() < mutation_prob:
            return self.__inversion_mutation(route)

        return route.copy()

    def __order_crossover(self, parent1, parent2):
        size = len(parent1)
        start, end = sorted(random.sample(range(1, size), 2))
        child = [None] * size
        # give child random section of parent1
        child[start:end + 1] = parent1[start:end + 1]
        # remaining parts of parent2
        # if those cities are not already in child #
        remaining = [
            city for city in parent2
            if city not in child
        ]
        # wrapping remaining cities correctly
        pos = list(range(end + 1, size)) + list(range(0, start))
        for pos, city in zip(pos, remaining):
            child[pos] = city

        # if child[0] != 0:
        #     raise ValueError("Child does not start on 0")

        return child

    def recombine(self, p1, p2, crossover_prob=1.0):
        if random.random() < crossover_prob:
            return (
                self.__order_crossover(p1, p2),
                self.__order_crossover(p1, p2)
            )
        return p1.copy(), p2.copy()

    # Setter for self.adj_matrix
    def set_adj_matrix(self, tsp_instance):
        self.adj_matrix = tsp_instance.copy()
        self.num_cities = len(self.adj_matrix)

    def evo_algo(
        self,
        pop_size=10,
        target_time=0.1,
        mutation_prob=0.7,
        crossover_prob=1.0
    ):
        start_time = time.time()
        if pop_size % 2 != 0 or pop_size < 2:
            raise ValueError("Population size must be even number of at leas
        self.populate(pop_size)
        print("Population Routes and Costs:")
        print("Generation 0:")
        # for route in self.population:
        #     print(route,"Cost:", self.evaluate_route(route))
        # use above loop instead to see all routes and costs
        best_route = min(self.population, key=self.evaluate_route)
        best_route = best_route.copy()
```

```
            best_cost = self.evaluate_route(best_route)
            print(f"Initial Best Cost: {best_cost}")
            print(f"Initial Best Route: {best_route}")

            generation = 0
            while time.time() - start_time < target_time:
                parents = self.select_parents(pop_size)
                offspring = []
                for i in range(0, pop_size, 2):
                    parent1 = parents[i]
                    parent2 = parents[i + 1]
                    child, child2 = self.recombine(parent1, parent2, crossover_p
                    mutate = self.mutate(child, mutation_prob)
                    mutate2 = self.mutate(child2, mutation_prob)
                    offspring.extend([mutate, mutate2])
                self.population = offspring
                for route in self.population:
                    cost = self.evaluate_route(route)
                    if cost < best_cost:
                        best_route = route.copy()
                        best_cost = cost
                        print(f"\nGeneration {generation}:")
                        print(f"Cost: {best_cost}")
                        print(f"Route: {best_route}")
                generation += 1
                # print(f"Generation {generation}:")
                # self.print_population()
                # use self.print_population instead to see all routes and costs

            print("Best Route:", best_route,"\nBest Cost:", best_cost)
            return best_route, best_cost

# copied from lab 1
# #
def build_adj_matrix_float(num_cities):
    adj_matrix = [
        [0.0 for _ in range(num_cities)]
        for _ in range(num_cities)
    ]
    return adj_matrix

def construct_costs_from_coords(adj_matrix, df, num_cities):
    for i in range(num_cities):
        for j in range(i + 1, num_cities):
            x_i = df.iloc[i]["x"]
            y_i = df.iloc[i]["y"]

            x_j = df.iloc[j]["x"]
            y_j = df.iloc[j]["y"]

            distance = float(
                np.sqrt((x_i - x_j) ** 2 + (y_i - y_j) ** 2)
            )

            adj_matrix[i][j] = distance
            adj_matrix[j][i] = distance
    return adj_matrix

# Change following CSV file if you want to use a different one
df = pd.read_csv('ulysses16.csv')
```

```
num_cities = len(df)

adj_matrix = build_adj_matrix_float(num_cities)
adj_matrix = construct_costs_from_coords(adj_matrix, df, num_cities)

im = InverseMutationTSP(5, 1, 50)
im.set_adj_matrix(adj_matrix)
# im.print_matrix()
im.evo_algo(pop_size=10, target_time=10, mutation_prob=0.7, crossover_prob=1.0)
```

```
Generating Routes ...
Population Routes and Costs:
Generation 0:
Initial Best Cost: 118.24272407538867
Initial Best Route: [0, 7, 8, 12, 6, 14, 11, 9, 2, 1, 15, 3, 5, 13, 4, 10]

Generation 3:
Cost: 112.13958886190957
Route: [10, 3, 7, 9, 8, 6, 11, 0, 13, 14, 12, 15, 1, 2, 5, 4]

Generation 6:
Cost: 108.92265205275233
Route: [6, 3, 0, 11, 8, 10, 4, 14, 12, 15, 7, 13, 5, 1, 2, 9]

Generation 6:
Cost: 105.76221112744138
Route: [9, 14, 10, 8, 11, 0, 3, 2, 1, 5, 13, 7, 15, 12, 4, 6]

Generation 16:
Cost: 104.9684081538217
Route: [1, 6, 10, 4, 11, 15, 5, 12, 14, 13, 9, 8, 7, 0, 3, 2]

Generation 17:
Cost: 91.83497840703447
Route: [9, 8, 7, 0, 3, 1, 2, 15, 5, 14, 6, 10, 4, 11, 12, 13]

Generation 116:
Cost: 82.94527013538296
Route: [12, 4, 10, 8, 6, 5, 14, 13, 15, 0, 3, 2, 1, 7, 9, 11]

Generation 710:
Cost: 82.54195791109777
Route: [9, 5, 13, 12, 11, 4, 10, 8, 6, 14, 7, 3, 1, 2, 0, 15]

Generation 6799:
Cost: 78.28248447685787
Route: [6, 11, 3, 1, 2, 0, 7, 15, 12, 9, 8, 10, 4, 14, 13, 5]

Generation 17532:
Cost: 77.4217205655024
Route: [15, 14, 13, 11, 12, 6, 5, 4, 10, 8, 9, 0, 7, 3, 2, 1]
Best Route: [15, 14, 13, 11, 12, 6, 5, 4, 10, 8, 9, 0, 7, 3, 2, 1]
Best Cost: 77.4217205655024
```

Out[39]: ([15, 14, 13, 11, 12, 6, 5, 4, 10, 8, 9, 0, 7, 3, 2, 1], 77.4217205655024)

Lab 5

Lab 5 is for comparison study between to Algorithm sample sizes. Imported/Read from a .csv file.

First we use central limit theory (CTL) to confirm our assumption's that sample sizes are sufficiently large for Welch's t-test, although even if a file fails here we do not remove it from the proceeding tests.

Next we compare the means between the two algorithms to see if they pass the null hypothesis or alternative hypothesis, in our case we are expecting algorithm A < algorithm B (which is alternative hypothesis). If a file/algorithm pair fail here then they do not proceed to the next test.

The final test is the Welch t-test conducted within Excel

hints for Excel:

=TTEST([DATA1],[DATA2],1,3) = p-value

Compare p-value to significance value

If the p-value is less than our significance level (α = 0.05), we can reject the null hypothesis

In [40]:
```
# Step 1:
# Confirming Welches T-Test Assumptions Using Central Limit Theory (CTL)
# #

import pandas as pd

# Using n>=30 for CLT sample size as rule of thumb
# From Lecture Slides
def check_welch_assumptions(file_list, min_sample_size=30):

    for file in file_list:
        df = pd.read_csv(file)
        print(f"\nChecking file: {file}")

        algorithm_A = df["Algorithm A"].dropna()
        algorithm_B = df["Algorithm B"].dropna()

        num_A = len(algorithm_A)
        num_B = len(algorithm_B)
        print(f"Algorithm A sample size: {num_A}")
        print(f"Algorithm B sample size: {num_B}")

        if num_A >= min_sample_size and num_B>= min_sample_size:
            print("Both sample sizes are sufficiently large")
        else:
            print("One or both of the sample sizes were not sufficiently large")

        print("-"*50)

file_list = [
    "data/problem1.csv",
    "data/problem2.csv",
    "data/problem3.csv",
```

```
    "data/problem4.csv",
    "data/problem5.csv",
]

check_welch_assumptions(file_list)
```

```
Checking file: data/problem1.csv
Algorithm A sample size: 100
Algorithm B sample size: 100
Both sample sizes are sufficiently large
--------------------------------------------------

Checking file: data/problem2.csv
Algorithm A sample size: 100
Algorithm B sample size: 100
Both sample sizes are sufficiently large
--------------------------------------------------

Checking file: data/problem3.csv
Algorithm A sample size: 100
Algorithm B sample size: 100
Both sample sizes are sufficiently large
--------------------------------------------------

Checking file: data/problem4.csv
Algorithm A sample size: 10
Algorithm B sample size: 10
One or both of the sample sizes were not sufficiently large
--------------------------------------------------

Checking file: data/problem5.csv
Algorithm A sample size: 100
Algorithm B sample size: 100
Both sample sizes are sufficiently large
--------------------------------------------------
```

Code finds data/problem4.csv sample size not sufficiently large enough for Welch's t-test according to CTL (using min_sample_size=30)

In [41]:
```
# Step 2: Formulating Your Hypotheses
# Want to use null and alternative hypothesis
# test if Algorithm A < Algorithm B, regarding
# average cost #

# Null Hypothesis (H0):
# Mean cost of Algorithm A >= Mean cost of Algorithm B

# Alternative Hypothesis (H1):
# Mean cost of Algorithm A < Mean cost of Algorithm B

# Step 3: Comparison of Means

def null_alt_hypo(file_list):

    for file in file_list:
        df = pd.read_csv(file)
        print(f"Checking file: {file}")

        algorithm_A = df["Algorithm A"].dropna()
        algorithm_B = df["Algorithm B"].dropna()
```

```
        num_A = len(algorithm_A)
        num_B = len(algorithm_B)

        mean_A = algorithm_A.mean()
        mean_B = algorithm_B.mean()

        if mean_A >= mean_B:
            print(f"{mean_A} >= {mean_B}")
            print("Passes Null Hypothesis (H0) check")
            print("No need to proceed to step 4.\n")
        if mean_A < mean_B:
            print(f"{mean_A} < {mean_B}")
            print("Passes Alternative Hypothesis (H0) check")
            print("Proceed to Step 4.\n")


file_list = [
    "data/problem1.csv",
    "data/problem2.csv",
    "data/problem3.csv",
    "data/problem4.csv",
    "data/problem5.csv",
]

null_alt_hypo(file_list)
```

```
Checking file: data/problem1.csv
46.34130133846998 < 50.296063268030004
Passes Alternative Hypothesis (H0) check
Proceed to Step 4.

Checking file: data/problem2.csv
47.6135082544 < 54.78352482838
Passes Alternative Hypothesis (H0) check
Proceed to Step 4.

Checking file: data/problem3.csv
55.70525981410001 >= 47.21986880534999
Passes Null Hypothesis (H0) check
No need to proceed to step 4.

Checking file: data/problem4.csv
34.0277800006 < 56.24540719099999
Passes Alternative Hypothesis (H0) check
Proceed to Step 4.

Checking file: data/problem5.csv
41.43093659835999 < 52.211609969310004
Passes Alternative Hypothesis (H0) check
Proceed to Step 4.
```

In [42]:
```
# Use following as example

a = 0.05

p_values = [
    0.156946991,#1
    0.038731927,#2
```

```
    0.039769132,#4
    0.002549866 #5
]

for p in p_values:
    if p < a:
        print("we reject null hypothesis")
    else:
        print("we accept hypothesis")

# checking previously failed problem3.csv

p = 0.018382283 #3

if p < a:
    print("we reject null hypothesis for p3")
else:
    print("we accept hypothesis for p3")
```

```
we accept hypothesis
we reject null hypothesis
we reject null hypothesis
we reject null hypothesis
we reject null hypothesis for p3
```